

LangChain Tools - Complete Notes with Examples

Contents

1	Introduction to Tools	3
1.1	Examples of Tools	3
2	Why Do We Need Tools?	3
3	Tool Architecture	3
4	Creating a Tool Using @tool Decorator	4
4.1	Example: Multiplication Tool	4
5	Tool Description Example	4
6	Viewing Tool Metadata	4
7	StructuredTool	4
8	Tool with Pydantic Schema	5
9	Binding Tools to LLM	5
10	Automatic Tool Calling	5
11	Multiple Tools Example	6
12	Tools with Agents	6
13	Weather API Tool Example	6
14	Database Tool Example	7
15	Vector Search Tool Example	7
16	Tool in RAG Applications	7
17	Tool Calling Workflow	8
18	Built-in Tools in LangChain	8
19	Real-World Projects Using Tools	8
19.1	1. AI Customer Support Bot	8
19.2	2. AI SQL Assistant	9
19.3	3. RAG Chatbot	9
19.4	4. AI Research Agent	9
19.5	5. AI Data Analyst	9

20 Interview Questions	9
20.1 What is a Tool in LangChain?	9
20.2 Difference Between Tool and Agent	9
20.3 Difference Between Tool and Retriever	9
20.4 What is @tool?	10
20.5 What is bind_tools()?	10
21 Summary	10

1 Introduction to Tools

A **Tool** in LangChain is a function that an LLM can call to interact with external systems and perform tasks beyond text generation.

1.1 Examples of Tools

- Web Search
- API Calls
- Database Queries
- File Operations
- Python Execution
- Vector Database Search
- Calculations

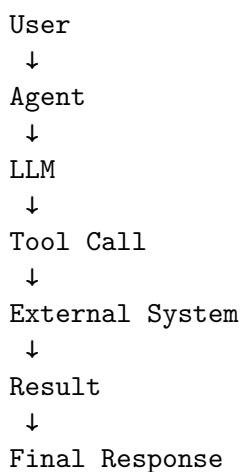
2 Why Do We Need Tools?

Large Language Models have limitations:

- Cannot access real-time information.
- Cannot query databases directly.
- Cannot interact with external APIs.
- May make mistakes in calculations.

Tools help overcome these limitations.

3 Tool Architecture



4 Creating a Tool Using @tool Decorator

4.1 Example: Multiplication Tool

```
from langchain.tools import tool

@tool
def multiply(a: int, b: int) -> int:
    """
    Multiply two numbers.
    """
    return a * b

result = multiply.invoke({
    "a": 10,
    "b": 20
})

print(result)
```

Output:

200

5 Tool Description Example

```
from langchain.tools import tool

@tool
def get_student_info(name: str):
    """
    Returns student details.
    """
    return f"{name} is enrolled in Python Course."
```

6 Viewing Tool Metadata

```
print(get_student_info.name)

print(get_student_info.description)

print(get_student_info.args)
```

Output:

```
get_student_info
Returns student details.
{'name': {'type': 'string'}}
```

7 StructuredTool

Used when a tool has multiple parameters.

```
from langchain.tools import StructuredTool

def add_numbers(a: int, b: int):
```

```

    return a + b

tool = StructuredTool.from_function(
    func=add_numbers,
    name="Adder",
    description="Adds two numbers"
)

```

Usage:

```

result = tool.invoke({
    "a": 10,
    "b": 20
})

print(result)

```

Output:

30

8 Tool with Pydantic Schema

```

from pydantic import BaseModel
from langchain.tools import tool

class CalculatorInput(BaseModel):
    a: int
    b: int

@tool(args_schema=CalculatorInput)
def multiply(a: int, b: int):
    """
    Multiply two numbers
    """
    return a * b

```

9 Binding Tools to LLM

```

from langchain_google_genai import ChatGoogleGenerativeAI

llm = ChatGoogleGenerativeAI(
    model="gemini-2.0-flash"
)

llm_with_tools = llm.bind_tools(
    [multiply]
)

```

10 Automatic Tool Calling

```

response = llm_with_tools.invoke(
    "What is 15 multiplied by 8?"
)

print(response)

```

The LLM decides whether to call the tool automatically.

11 Multiple Tools Example

```
from langchain.tools import tool

@tool
def add(a:int,b:int):
    return a+b

@tool
def subtract(a:int,b:int):
    return a-b

@tool
def multiply(a:int,b:int):
    return a*b
```

Binding Multiple Tools:

```
llm_with_tools = llm.bind_tools(
    [add, subtract, multiply]
)
```

12 Tools with Agents

Agents can decide which tool to use.

```
from langchain.agents import initialize_agent
from langchain.agents import AgentType

agent = initialize_agent(
    tools=[add, multiply],
    llm=llm,
    agent=AgentType.ZERO_SHOT_REACT_DESCRIPTION,
    verbose=True
)
```

Running Agent:

```
agent.run(
    "Multiply 10 by 5 and then add 20"
)
```

Output:

70

13 Weather API Tool Example

```
from langchain.tools import tool
import requests

@tool
def get_weather(city: str):
    """
    Get weather information.
    """
```

```

response = requests.get(
    f"https://api.weatherapi.com/..."
)

return response.json()

```

14 Database Tool Example

```

from langchain.tools import tool
import sqlite3

@tool
def get_students():

    conn = sqlite3.connect("students.db")

    data = conn.execute(
        "SELECT * FROM students"
    ).fetchall()

    conn.close()

    return data

```

15 Vector Search Tool Example

```

@tool
def search_documents(query:str):

    docs = vectorstore.similarity_search(
        query,
        k=3
    )

    return docs

```

16 Tool in RAG Applications

```

User Query
  ↓
Agent
  ↓
Retriever Tool
  ↓
Vector Database
  ↓
Relevant Documents
  ↓
LLM
  ↓
Final Answer

```

Example:

```
@tool
def retrieve_docs(query:str):

    docs = vectorstore.similarity_search(
        query
    )

    return docs
```

17 Tool Calling Workflow

```
User
↓
LLM
↓
Need External Information?
↓
Yes
↓
Tool Call
↓
Result
↓
LLM
↓
Final Answer
```

18 Built-in Tools in LangChain

Tool	Purpose
Tavily Search	Internet Search
DuckDuckGo Search	Web Search
Wikipedia Tool	Wikipedia Search
Python REPL Tool	Execute Python Code
SQL Tool	Database Querying
Retriever Tool	Vector Search
Shell Tool	Execute Commands
Requests Tool	API Calls

19 Real-World Projects Using Tools

19.1 1. AI Customer Support Bot

Tools Used:

- FAQ Retriever
- CRM API
- Ticket Database

19.2 2. AI SQL Assistant

Tools Used:

- MySQL
- PostgreSQL
- SQLite

19.3 3. RAG Chatbot

Tools Used:

- FAISS
- ChromaDB
- Pinecone

19.4 4. AI Research Agent

Tools Used:

- Tavily Search
- Wikipedia
- Arxiv

19.5 5. AI Data Analyst

Tools Used:

- Python REPL
- Pandas
- SQL Database

20 Interview Questions

20.1 What is a Tool in LangChain?

A Tool is a callable function that allows an LLM or Agent to interact with external systems such as APIs, databases, files, search engines, and vector stores.

20.2 Difference Between Tool and Agent

Tool	Agent
Performs a specific task	Makes decisions
Executes actions	Chooses tools
Function	Orchestrator

20.3 Difference Between Tool and Retriever

Tool	Retriever
Generic Action	Retrieves Documents
Can call APIs, DBs	Used in RAG
Broader Concept	Specialized Tool

20.4 What is @tool?

A decorator that converts a Python function into a LangChain Tool.

20.5 What is bind_tools()?

Used to attach tools to an LLM so that the model can decide when to call them.

21 Summary

- Tools connect LLMs with external systems.
- Agents use tools to solve complex tasks.
- Tools are essential for RAG, AI Assistants, SQL Agents, and Automation Systems.
- LangChain provides built-in tools and supports custom tool creation.